Solid State Physics
Tight-Binding Model Project

Zengxuan Liu
3042027511

April 30, 2026

Contents

Chapter 1 **Simple 1D Tight-Binding Model** — Page 2

1.1 Task 1: Hamiltonian construction 2

1.2 Task 2: Eigenvalue problem 3

Chapter 2 **The Su–Schrieffer–Heeger (SSH) model** — Page 6

2.1 Task 1: Hamiltonian construction 6

2.2 Task 2: Bulk band structure 7

2.3 Task 3: Edge states 8

2.4 Task 4: Robustness of topological edge states 9

Chapter **Appendix** — Page 12

Chapter 1

Simple 1D Tight-Binding Model

1.1 Task 1: Hamiltonian construction

Question 1

Write a program that constructs the Hamiltonian matrix in the periodic boundary conditions (PBC), where site N is identified with site 0.

Question 2

Modify the code to handle open boundary conditions (OBC), where sites 0 and $N-1$ are not connected.

Definition 1.1.1: Hamiltonian

The Hamiltonian for the 1D tight-binding model is given by:

$$H = -t \sum_n \Big(|n+1\rangle \langle n| + |n\rangle \langle n+1| \Big)$$

Solution: We are going to define the Hamiltonian matrix for the 1D tight-binding model under both periodic and open boundary conditions using the code block below:

```
# PBC
def pbc_Hamiltonian(n, t = 1):
    H = np.zeros((n, n))
    for i in range(n):
        H[i, (i + 1) % n] = -t
        H[(i + 1) % n, i] = -t
    return H

# OBC
def obc_Hamiltonian(n, t = 1):
    H = np.zeros((n, n))
    for i in range(n - 1):
        H[i, i + 1] = -t
        H[i + 1, i] = -t
    return H
```

Note:-

n is the number of lattice sites and t is the hopping parameter. The key to ensure periodic boundary conditions is to use modulo arithmetic to connect the first and last lattice sites.

Example 1.1.1 (Simple 1D Tight-Binding Model Hamiltonian in PBC and OBC)

```
# Example
N = 5
H_PBC = pbc_Hamiltonian(N)
H_OBC = obc_Hamiltonian(N)

print("Hamiltonian in PBC:")
print(H_PBC)
print(scipy.linalg.ishermitian(H_PBC))
print("Hamiltonian in OBC:")
print(H_OBC)
print(scipy.linalg.ishermitian(H_OBC))
```

```
Hamiltonian in PBC:
[[ 0. -1.  0.  0. -1.]
[-1.  0. -1.  0.  0.]
[ 0. -1.  0. -1.  0.]
[ 0.  0. -1.  0. -1.]
[-1.  0.  0. -1.  0.]]
True
Hamiltonian in OBC:
[[ 0. -1.  0.  0.  0.]
[-1.  0. -1.  0.  0.]
[ 0. -1.  0. -1.  0.]
[ 0.  0. -1.  0. -1.]
[ 0.  0.  0. -1.  0.]]
True
```

1.2 Task 2: Eigenvalue problem

Question 1

Diagonalize your Hamiltonian for both boundary conditions for $N = 100$. It is convenient to use `numpy.linalg.eigh()`.

Solution: Using `np.linalg.eigh()` to diagonalize the Hamiltonian:

```
# Diagonalize
N = 100
E_PBC, psi_PBC = np.linalg.eigh(pbc_Hamiltonian(N))
E_OBC, psi_OBC = np.linalg.eigh(obc_Hamiltonian(N))
```

Note:-

The outputs of `np.linalg.eigh()` are arrays of eigenvalues and respective eigenvectors, and the arrays are sorted in ascending order of eigenvalues.

Question 2

Plot the energy spectrum of both the periodic and open systems on the same figure. How similar or different are they? Is there a degeneracy in the periodic spectrum? What is the origin of this degeneracy? Does the open system break or preserve this degeneracy? Why?

Solution: Plotting the energy spectrum:

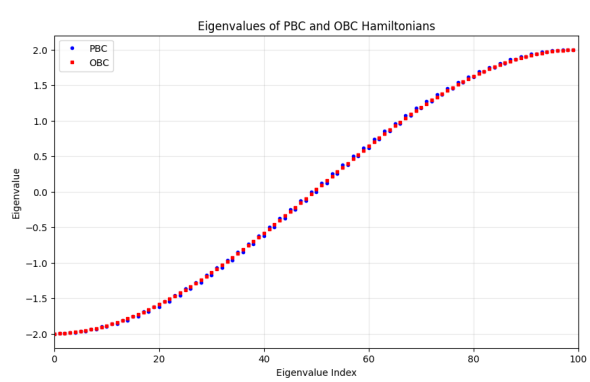

Figure 1.1: Eigenvalues of PBC and OBC Hamiltonians

Note:-

Consider the Hamiltonian and eigenvalue equation for a 1D tight-binding model:

$$H = -t \sum_{n=1}^{N} \Big(|n+1\rangle \langle n| + |n\rangle \langle n+1| \Big), \; H|\psi\rangle = E|\psi\rangle$$

where $|\psi\rangle = \sum_{n=1}^{N} \psi_n |n\rangle$. Furthermore, we can express the eigenequation as:

$$E\psi_n = -t(\psi_{n+1} + \psi_{n-1})$$

(a) PBC: As common choice for periodic system, we can use plane wave eigenstates, we can write:

$$\psi_n = \frac{1}{\sqrt{N}} e^{ikn}$$

where $k = \frac{2\pi m}{N}$ for $m = 0, 1, \ldots, N-1$. Thus, we can substitute this into the eigenequation and find the corresponding eigenvalues:

$$E(k) = -2t\cos(k) = -2t\cos\left(\frac{2\pi m}{N}\right)$$

which shows the degeneracy that $E(k) = E(-k)$.

(b) OBC: For the open boundary conditions, we can use standing wave eigenstates, we can write:

$$\psi_n = \sqrt{\frac{2}{N+1}} \sin\left(\frac{m\pi n}{N+1}\right)$$

where $m = 1, 2, \ldots, N$. Thus, we can substitute this into the eigenequation and find the corresponding eigenvalues:

$$E(m) = -2t\cos\left(\frac{m\pi}{N+1}\right)$$

- Both systems have very similar spectra for $N = 100$.
- There is a degeneracy in the periodic system because of periodic boundary conditions. This periodicity maintains the translational symmetry which ensures that each atom has the same neighborhood environment. Therefore, this symmetry leads to degenerate energy levels.
- The open system breaks this degeneracy because the boundary conditions are different, leading to a unique energy spectrum for each state.

Question 3

1. Plot the wave func1on probability distribu1on, $|\psi(x)|^2$, for the ground state and first two excited states of the **periodic** system as a func1on of posi1on and show them on the same plot. What do you notice about the first two excited states?
2. Plot the wave func1on probability distribu1on, $|\psi_m(x)|^2$, for the ground state and first two excited states of the **open** system as a func1on of posi1on and show them on the same plot. What do you notice about the number of nodes?

Solution: For 1D chains, the position can be represented by a discrete set of sites. Here, the position is indexed by n. Plotting the wave function probability distribution:

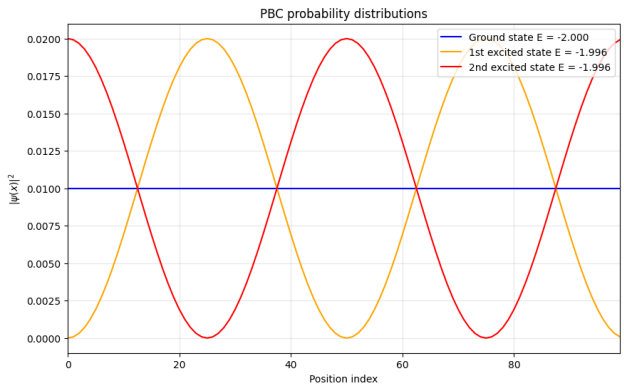

Figure 1.2: Probability distribution for PBC system

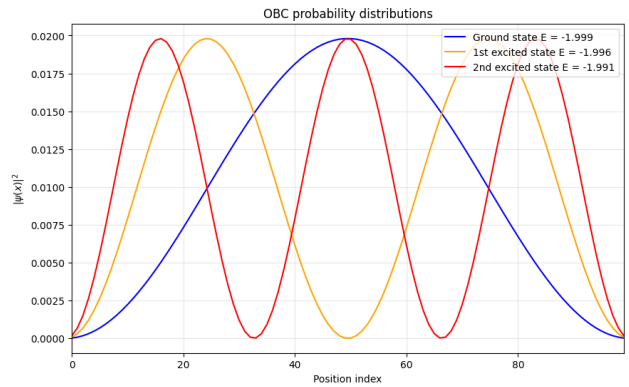

Figure 1.3: Probability distribution for OBC system

Note:-

1. PBC: The probability distribution for the ground state and first two excited states shows a periodic pattern due to the boundary conditions. For the first two excited states, they are degenerate and the numerical eigenvectors are different linear combinations of the degenerate states. But both of them show two nodes, and their shapes are quite alike.

2. OBC: The number of nodes in the probability distribution for the ground state and first two excited states are 0, 1, and 2 respectively, increasing with the energy level.

Chapter 2

The Su–Schrieffer–Heeger (SSH) model

2.1 Task 1: Hamiltonian construction

Question 1

Write a program to construct the Hamiltonian matrix for this system in both periodic and open boundary conditions. Make sure that there is an equal number of A and B sites.

Definition 2.1: The Su–Schrieffer–Heeger (SSH) model

The Su-Schrieffer-Heeger model describes a dimerized chain of N unit cells with alterna1ng hopping magnitudes. The unit cell contains two sites (A and B sites), with intracell hopping v and intercell hopping ω. The Hamiltonian is given by

$$H = -\sum_{n=0} \Big(v\,|n;B\rangle\langle n;A| + v\,|n;A\rangle\langle n;B| + \omega\,|n+1;A\rangle\langle n;B| + \omega\,|n;B\rangle\langle n+1;A| \Big)$$

where n runs over all unit cells. In what follows, use $\omega = 1$ in your simulation.

Solution: We are going to define the Hamiltonian matrix for the SSH model in both periodic and open boundary conditions using the code block below:

```
# PBC
def pbc_Hamiltonian(n, nu, omega = 1):
    H = np.zeros((2 * n, 2 * n))
    for i in range(n):
        H[2 * i + 1, 2 * i] = -nu
        H[2 * i, 2 * i + 1] = -nu
        H[2 * (i + 1) % (2 * n), 2 * i + 1] = -omega
        H[2 * i + 1, 2 * (i + 1) % (2 * n)] = -omega
    return H

# OBC
def obc_Hamiltonian(n, nu, omega=1):
    H = np.zeros((2 * n, 2 * n))
    for i in range(n):
        H[2 * i + 1, 2 * i] = -nu
        H[2 * i, 2 * i + 1] = -nu
    for i in range(n - 1):
        H[2 * (i + 1), 2 * i + 1] = -omega
        H[2 * i + 1, 2 * (i + 1)] = -omega
    return H
```

Example 2.1 (SSH Hamiltonian in PBC and OBC)

```python
# Example
N = 3
nu = 2
omega = 1
H_PBC = pbc_Hamiltonian(N, nu, omega)
H_OBC = obc_Hamiltonian(N, nu, omega)

print("Hamiltonian␣in␣PBC:")
print(H_PBC)
print(scipy.linalg.ishermitian(H_PBC))
print("Hamiltonian␣in␣OBC:")
print(H_OBC)
print(scipy.linalg.ishermitian(H_OBC))
```

```
Hamiltonian in PBC:
[[ 0. -2.  0.  0.  0. -1.]
 [-2.  0. -1.  0.  0.  0.]
 [ 0. -1.  0. -2.  0.  0.]
 [ 0.  0. -2.  0. -1.  0.]
 [ 0.  0.  0. -1.  0. -2.]
 [-1.  0.  0.  0. -2.  0.]]
True
Hamiltonian in OBC:
[[ 0. -2.  0.  0.  0.  0.]
 [-2.  0. -1.  0.  0.  0.]
 [ 0. -1.  0. -2.  0.  0.]
 [ 0.  0. -2.  0. -1.  0.]
 [ 0.  0.  0. -1.  0. -2.]
 [ 0.  0.  0.  0. -2.  0.]]
True
```

2.2 Task 2: Bulk band structure

Question 1

- Numerically calculate the band structure for the periodic system for $N = 100$ for different ratios of $\nu/\omega = \{0.5, 1, 2\}$.
- Which parameters correspond to a metal and which correspond to an insulator?
- Plot the numerical band structures for the different parameters of ν/ω.

Solution: Calculate band structure for the periodic system for $N = 100$ and different values of ν/ω and plot the numerical band structures:

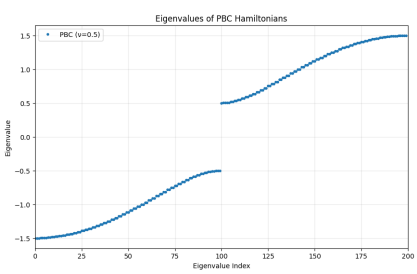

Figure 2.1: SSH PBC energy spectrum for $\nu/\omega = 0.5$

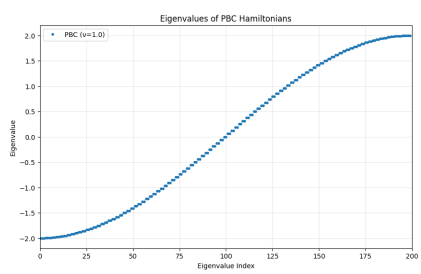

Figure 2.2: SSH PBC energy spectrum for $\nu/\omega = 1.0$

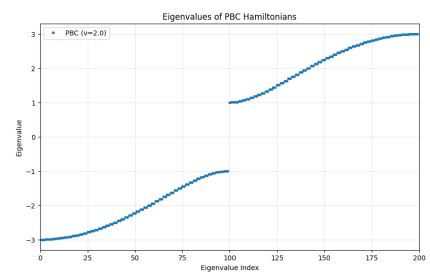

Figure 2.3: SSH PBC energy spectrum for $\nu/\omega = 2.0$

Note:-

$\nu = 1$ corresponds to a metal while $\nu = 0.5$ and $\nu = 2.0$ correspond to insulators. Because the energy bands are gapless for $\nu = 1$ and gapped for $\nu = 0.5$ and $\nu = 2.0$.

2.3 Task 3: Edge states

Question 1

Construct the OBC Hamiltonian of the SSH model for $N = 50$ and plot the energy spectrum for different ratios of $\nu/\omega = \{0.5, 1, 2\}$.

Solution: Calculate the energy spectrum for the open system for $N = 50$ and different values of ν/ω and plot the energy spectrum:

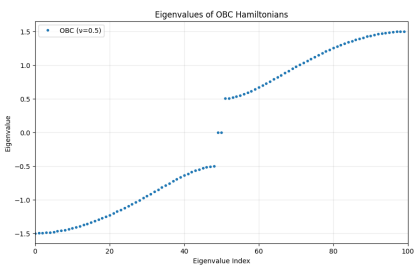

Figure 2.4: SSH OBC energy spectrum for $\nu/\omega = 0.5$

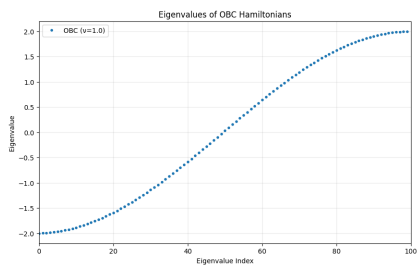

Figure 2.5: SSH OBC energy spectrum for $\nu/\omega = 1.0$

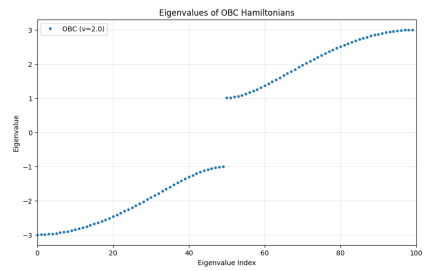

Figure 2.6: SSH OBC energy spectrum for $\nu/\omega = 2.0$

Question 2

- Identify the zero-energy states for the case when $\nu/\omega = 0.5$. How many are they?
- Plot the zero-energy states wavefunctions and verify that they are localized at the boundaries, hence the name 'edge' states.

Solution: Identify the zero-energy states for the case when $\nu/\omega = 0.5$ using threshold $\epsilon = 1e - 10$, and plot the wave functions of the zero-energy states:

```
zero_states = np.where(np.abs(E_OBC_1) < 1e-10)[0]
print("Zero-energy state indices:", zero_states)
print("Number of zero-energy states:", len(zero_states))
print("Energies:", E_OBC_1[zero_states])
```

The outputs are as follows:

```
Zero-energy state indices: [49 50]
Number of zero-energy states: 2
Energies: [-9.54080635e-16  3.78186994e-16]
```

Note:-

The outputs show that there are two zero-energy states for the case when $\nu/\omega = 0.5$.

Plotting the wave functions of the zero-energy states:

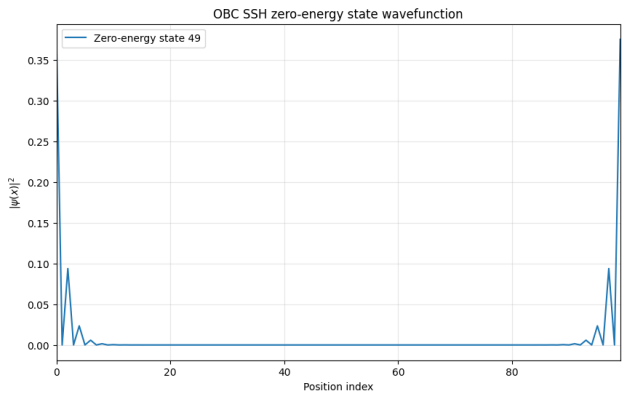

Figure 2.7: Zero-energy state wavefunction 1

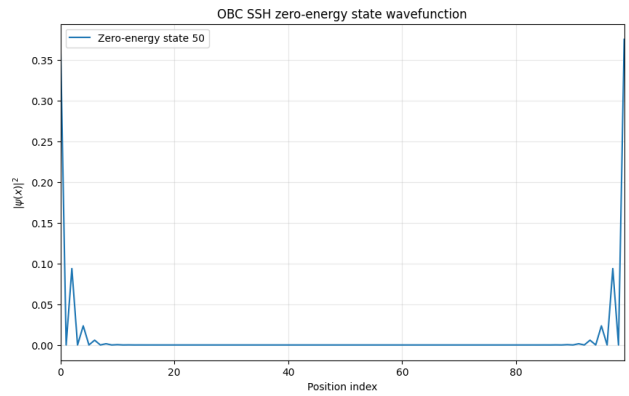

Figure 2.8: Zero-energy state wavefunction 2

Note:-

As shown in the plots, the wavefuncitons of zero-energy state are quite localized at the boundaries, which confirms that they are indeed edge states.

2.4 Task 4: Robustness of topological edge states

Question 1

- Add a weak random on-site disorder to your OBC SSH Hamiltonian,

$$H_{\text{disorder}} = \sum_n \epsilon_{nA} \left|n; A\right\rangle \left\langle n; A\right| + \epsilon_{nB} \left|n; B\right\rangle \left\langle n; B\right|$$

  where $\epsilon_{nA,B}$ are random variables drawn from a uniform distribu1on in the range $[-W, W]$.

- Show that the zero-energy localized states persist for small noise amplitudes ($W < |\omega - \nu|$) by plotting the spectrum and their wave function probability distribu1ons. For concreteness, use $\nu = 0.5$ and $W = \{0.2, 0.4\}$.

Solution: Construct the disordered Hamiltonian by adding random on-site disorder to the OBC SSH Hamiltonian:

```
# OBC Hamiltonian for SSH model with disorder
def obc_Hamiltonian_disordered(n, W, nu, omega=1, seed=None):
    H = np.zeros((2 * n, 2 * n))
    rng = np.random.default_rng(seed)
    # SSH Hamiltonian
    for i in range(n):
        H[2 * i + 1, 2 * i] = -nu
        H[2 * i, 2 * i + 1] = -nu
    for i in range(n - 1):
        H[2 * (i + 1), 2 * i + 1] = -omega
        H[2 * i + 1, 2 * (i + 1)] = -omega
    # Disorder term
    for i in range(n):
        H[2 * i, 2 * i] = rng.uniform(-W, W)
        H[2 * i + 1, 2 * i + 1] = rng.uniform(-W, W)

    return H
```

Calculate the energy spectrum and wave function probability distributions for the disordered system using the random seed of 67 for reproducibility:

```
W1 = 0.2
W2 = 0.4
nu = 0.5
N = 100

E1, psi1 = np.linalg.eigh(obc_Hamiltonian_disordered(N, W1, nu, seed=67))
E2, psi2 = np.linalg.eigh(obc_Hamiltonian_disordered(N, W2, nu, seed=67))
```

Then we can find the edge states by identifying the eigenvalues that are closest to zero:

```
edge_states_1 = np.argsort(np.abs(E1))[:2]
edge_states_2 = np.argsort(np.abs(E2))[:2]

print("W␣=␣0.2␣edge␣state␣energies:", E1[edge_states_1])
print("W␣=␣0.4␣edge␣state␣energies:", E2[edge_states_2])
```

The outputs are as follows:

```
W = 0.2 edge state energies: [-0.02428976  0.02461507]
W = 0.4 edge state energies: [-0.04884587  0.05275579]
```

Note:-

The outputs show that the same edge states in the clean system persist in the presence of disorder, albeit with slightly shifted energies.

Plotting the energy spectrum and wave function probability distributions for the disordered system:

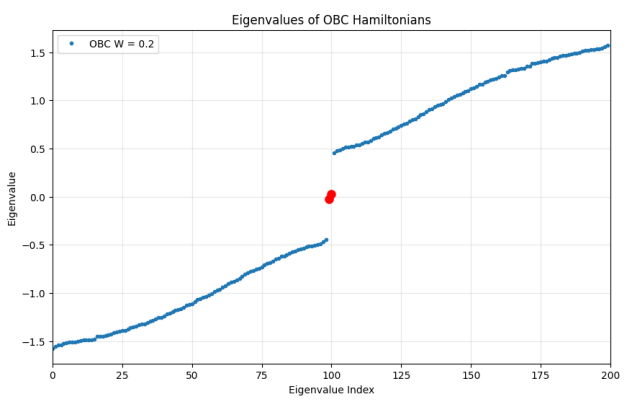

Figure 2.9: Energy spectrum for OBC with disorder $W = 0.2$

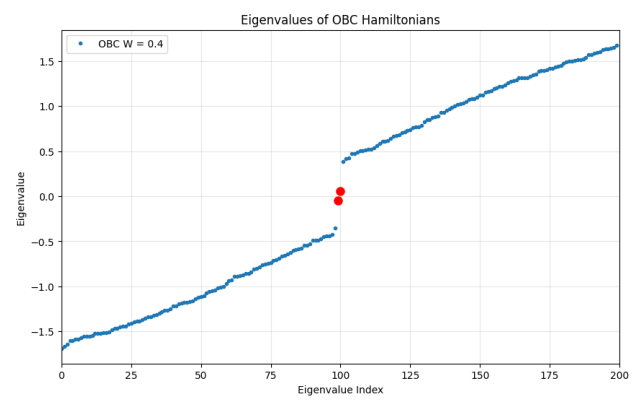

Figure 2.10: Energy spectrum for OBC with disorder $W = 0.4$

Note:-

These plots shows that bigger disorder strength leads to larger energy shifts for the edge states, but they still persist as long as the disorder strength is smaller than the bulk gap.

Plotting the wave function probability distributions for the edge states in the disordered system:

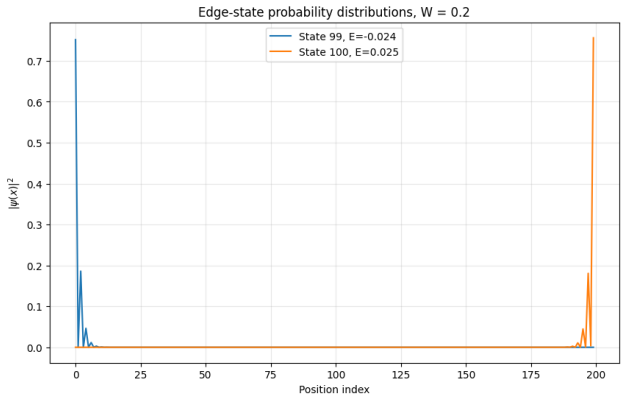

Figure 2.11: Edge state probability distribution for $W = 0.2$

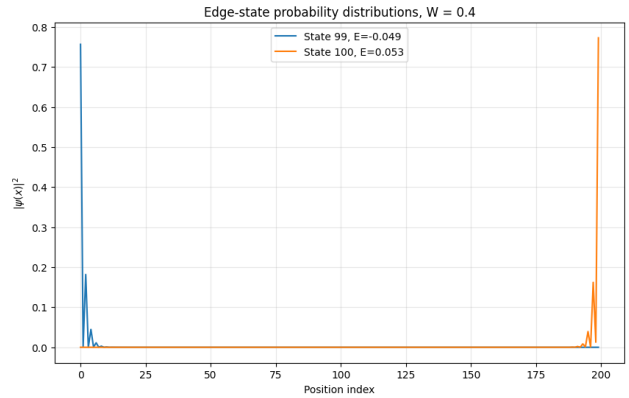

Figure 2.12: Edge state probability distribution for $W = 0.4$

Appendix

I used Jupyter Notebook to finish the coding of this project and have uploaded the notebook to GitHub. You can find it at the following link https://github.com/zengxuanliu2005/Solid_State_Physics_Project. Also my git commit history is available at the same repository.
Complete code for this project will be provided as follows for reference. Since I use Jupyter Notebook to finish the coding of this project, all codes will be presented here in the form of cells.

Importing necessary libraries

```
import numpy as np
import matplotlib.pyplot as plt
import scipy
```

Simple tight-binding model code

Task 1: Hamiltonian construction

```
# PBC
def pbc_Hamiltonian(n, t = 1):
    H = np.zeros((n, n))
    for i in range(n):
        H[i, (i + 1) % n] = -t
        H[(i + 1) % n, i] = -t
    return H

# OBC
def obc_Hamiltonian(n, t = 1):
    H = np.zeros((n, n))
    for i in range(n - 1):
        H[i, i + 1] = -t
        H[i + 1, i] = -t
    return H
```

```
# Example
N = 5
H_PBC = pbc_Hamiltonian(N)
H_OBC = obc_Hamiltonian(N)

print("Hamiltonian␣in␣PBC:")
print(H_PBC)
print(scipy.linalg.ishermitian(H_PBC))
print("Hamiltonian␣in␣OBC:")
print(H_OBC)
print(scipy.linalg.ishermitian(H_OBC))
```

Task 2: Eigenvalue problem

```
# Diagonalize
N = 100
E_PBC, psi_PBC = np.linalg.eigh(pbc_Hamiltonian(N))
E_OBC, psi_OBC = np.linalg.eigh(obc_Hamiltonian(N))
```

```
plt.figure(figsize=(10, 6))
N = 100

plt.plot(E_PBC, 'o', label='PBC', markersize=3, color='blue')
plt.plot(E_OBC, 's', label='OBC', markersize=3, color='red')
plt.xlabel('Eigenvalue␣Index')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues␣of␣PBC␣and␣OBC␣Hamiltonians')
plt.legend(loc='upper␣left')
plt.grid(True, alpha = 0.3)
plt.xlim(0, N)
plt.show()
```

```
# Ground state and first two excited state probability plots
def plot_ground_and_excited_states(N, E, psi, title):
    x = np.arange(N)

    psi_gs = psi[:, 0]   # ground state
    psi_e1 = psi[:, 1]   # first excited state
    psi_e2 = psi[:, 2]   # second excited state

    plt.figure(figsize=(10, 6))
    plt.plot(x, np.abs(psi_gs)**2, label=f"{'Ground␣state':}␣E␣=␣{E[0]:.3f}", color="blue")
    plt.plot(x, np.abs(psi_e1)**2, label=f"{'1st␣excited␣state':}␣E␣=␣{E[1]:.3f}", color="orange")
    plt.plot(x, np.abs(psi_e2)**2, label=f"{'2nd␣excited␣state':}␣E␣=␣{E[2]:.3f}", color="red")

    plt.xlabel("Position␣index")
    plt.ylabel(r"$|\psi(x)|^2$")
    plt.xlim(0, N-1)
    plt.title(title)
    plt.legend(loc="upper␣right")
    plt.grid(True, alpha=0.3)
    plt.show()

plot_ground_and_excited_states(N, E_PBC, psi_PBC, "PBC␣probability␣distributions")
plot_ground_and_excited_states(N, E_OBC, psi_OBC, "OBC␣probability␣distributions")
```

SSH model code

Task 1: Hamiltonian construction

```
# PBC
def pbc_Hamiltonian(n, nu, omega = 1):
    H = np.zeros((2 * n, 2 * n))
    for i in range(n):
        H[2 * i + 1, 2 * i] = -nu
        H[2 * i, 2 * i + 1] = -nu
        H[2 * (i + 1) % (2 * n), 2 * i + 1] = -omega
        H[2 * i + 1, 2 * (i + 1) % (2 * n)] = -omega
    return H
```

```
# OBC
def obc_Hamiltonian(n, nu, omega=1):
    H = np.zeros((2 * n, 2 * n))
    for i in range(n):
        H[2 * i + 1, 2 * i] = -nu
        H[2 * i, 2 * i + 1] = -nu
    for i in range(n - 1):
        H[2 * (i + 1), 2 * i + 1] = -omega
        H[2 * i + 1, 2 * (i + 1)] = -omega
    return H
```

Task 2: Bulk band structure

```
# Diagonalize
N = 100
nu_1 = 0.5
nu_2 = 1
nu_3 = 2
omega = 1
E_PBC_1, psi_PBC_1 = np.linalg.eigh(pbc_Hamiltonian(N, nu_1, omega))
E_PBC_2, psi_PBC_2 = np.linalg.eigh(pbc_Hamiltonian(N, nu_2, omega))
E_PBC_3, psi_PBC_3 = np.linalg.eigh(pbc_Hamiltonian(N, nu_3, omega))
```

```
N = 100
plt.figure(figsize=(10, 6))

plt.plot(E_PBC_1, 'o', markersize=3, label='PBC␣ (=0.5)')
plt.xlabel('Eigenvalue␣Index')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues␣of␣PBC␣Hamiltonians')
plt.legend(loc='upper␣left')
plt.grid(True, alpha = 0.3)
plt.xlim(0, 2 * N)
plt.show()

plt.figure(figsize=(10, 6))

plt.plot(E_PBC_2, 'o', markersize=3, label='PBC␣ (=1.0)')
plt.xlabel('Eigenvalue␣Index')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues␣of␣PBC␣Hamiltonians')
plt.legend(loc='upper␣left')
plt.grid(True, alpha = 0.3)
plt.xlim(0, 2 * N)
plt.show()

plt.figure(figsize=(10, 6))

plt.plot(E_PBC_3, 'o', markersize=3, label='PBC␣ (=2.0)')
plt.xlabel('Eigenvalue␣Index')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues␣of␣PBC␣Hamiltonians')
plt.legend(loc='upper␣left')
plt.grid(True, alpha = 0.3)
plt.xlim(0, 2 * N)
plt.show()
```

Task 3: Edge states

```
# Diagonalize
N = 50
nu1 = 0.5
nu2 = 1
nu3 = 2
omega = 1
E_OBC_1, psi_OBC_1 = np.linalg.eigh(obc_Hamiltonian(N, nu1, omega))
E_OBC_2, psi_OBC_2 = np.linalg.eigh(obc_Hamiltonian(N, nu2, omega))
E_OBC_3, psi_OBC_3 = np.linalg.eigh(obc_Hamiltonian(N, nu3, omega))
```

```
plt.figure(figsize=(10, 6))
N = 50

plt.plot(E_OBC_1, 'o', markersize=3, label='OBC␣ (=0.5)')
plt.xlabel('Eigenvalue␣Index')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues␣of␣OBC␣Hamiltonians')
plt.legend(loc='upper␣left')
plt.grid(True, alpha = 0.3)
plt.xlim(0, 2 * N)
plt.show()

plt.figure(figsize=(10, 6))

plt.plot(E_OBC_2, 'o', markersize=3, label='OBC␣ (=1.0)')
plt.xlabel('Eigenvalue␣Index')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues␣of␣OBC␣Hamiltonians')
plt.legend(loc='upper␣left')
plt.grid(True, alpha = 0.3)
plt.xlim(0, 2 * N)
plt.show()

plt.figure(figsize=(10, 6))

plt.plot(E_OBC_3, 'o', markersize=3, label='OBC␣ (=2.0)')
plt.xlabel('Eigenvalue␣Index')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues␣of␣OBC␣Hamiltonians')
plt.legend(loc='upper␣left')
plt.grid(True, alpha = 0.3)
plt.xlim(0, 2 * N)
plt.show()
```

```
zero_states = np.where(np.abs(E_OBC_1) < 1e-10)[0]
print("Zero-energy␣state␣indices:", zero_states)
print("Number␣of␣zero-energy␣states:", len(zero_states))
print("Energies:", E_OBC_1[zero_states])
```

```
x = np.arange(2 * N)
for i in zero_states:
    prob = np.abs(psi_OBC_1[:, i])**2
    plt.figure(figsize=(10, 6))
    plt.plot(x, prob, label=f"Zero-energy␣state␣{i}")

    plt.xlabel("Position␣index")
    plt.ylabel(r"$|\psi(x)|^2$")
    plt.title("OBC␣SSH␣zero-energy␣state␣wavefunction")
    plt.legend(loc="upper␣left")
```

```
    plt.grid(True, alpha=0.3)
    plt.xlim(0, 2 * N - 1)
    plt.show()
```

Task 4: Robustness of topological edge states

```
# OBC Hamiltonian for SSH model with disorder
def obc_Hamiltonian_disordered(n, W, nu, omega=1, seed=None):
    H = np.zeros((2 * n, 2 * n))
    rng = np.random.default_rng(seed)
    # SSH Hamiltonian
    for i in range(n):
        H[2 * i + 1, 2 * i] = -nu
        H[2 * i, 2 * i + 1] = -nu
    for i in range(n - 1):
        H[2 * (i + 1), 2 * i + 1] = -omega
        H[2 * i + 1, 2 * (i + 1)] = -omega
    # Disorder term
    for i in range(n):
        H[2 * i, 2 * i] = rng.uniform(-W, W)
        H[2 * i + 1, 2 * i + 1] = rng.uniform(-W, W)

    return H
```

```
W1 = 0.2
W2 = 0.4
nu = 0.5
N = 100

E1, psi1 = np.linalg.eigh(obc_Hamiltonian_disordered(N, W1, nu, seed=67))
E2, psi2 = np.linalg.eigh(obc_Hamiltonian_disordered(N, W2, nu, seed=67))
```

```
edge_states_1 = np.argsort(np.abs(E1))[:2]
edge_states_2 = np.argsort(np.abs(E2))[:2]

print("W␣=␣0.2␣edge␣state␣energies:", E1[edge_states_1])
print("W␣=␣0.4␣edge␣state␣energies:", E2[edge_states_2])
```

```
plt.figure(figsize=(10, 6))
N = 100

plt.plot(E1, 'o', markersize=3, label="OBC␣W␣=␣0.2")
plt.plot(edge_states_1, E1[edge_states_1], "ro", markersize=8)
plt.xlabel('Eigenvalue␣Index')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues␣of␣OBC␣Hamiltonians')
plt.legend(loc='upper␣left')
plt.grid(True, alpha = 0.3)
plt.xlim(0, 2 * N)
plt.show()

plt.figure(figsize=(10, 6))

plt.plot(E2, 'o', markersize=3, label="OBC␣W␣=␣0.4")
plt.plot(edge_states_2, E2[edge_states_2], "ro", markersize=8)
plt.xlabel('Eigenvalue␣Index')
plt.ylabel('Eigenvalue')
plt.title('Eigenvalues␣of␣OBC␣Hamiltonians')
```

```
plt.legend(loc='upper␣left')
plt.grid(True, alpha = 0.3)
plt.xlim(0, 2 * N)
plt.show()
```

```
x = np.arange(2 * N)

for W, E, psi, edge_states in [
    (0.2, E1, psi1, edge_states_1),
    (0.4, E2, psi2, edge_states_2),
]:
    plt.figure(figsize=(10, 6))

    for idx in edge_states:
        plt.plot(x, np.abs(psi[:, idx])**2, label=f"State␣{idx},␣E={E[idx]:.3f}")

    plt.xlabel("Position␣index")
    plt.ylabel(r"$|\psi(x)|^2$")
    plt.title(f"Edge-state␣probability␣distributions,␣W␣=␣{W}")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()
```